

Conversione dei tipi

Conversione dei tipi

Come trasformare un valore da un tipo all'altro in Python.

Perché convertire i tipi?

Immagina di voler stampare questo messaggio: `"Hai 16 anni"`. Il numero `16` è un intero, ma per metterlo in mezzo al testo devi trasformarlo in una stringa di testo. Senza questa trasformazione, Python si ferma con un errore.

Questa operazione — trasformare un valore da un tipo all'altro — si chiama **conversione di tipo**.

Conversione automatica (implicita)

A volte Python fa la conversione da solo, senza che tu debba fare niente. Succede quando sommi un numero intero con uno decimale: Python trasforma automaticamente l'intero in decimale per poter fare il calcolo.

```
intero = 5
decimale = 2.5
risultato = intero + decimale

print(risultato)          # 7.5
print(type(risultato))    # <class 'float'>
```

Python fa questo solo quando è sicuro di non perdere informazioni.

Conversione manuale (esplicita)

Altre volte devi essere tu a dire a Python di convertire. Si usano funzioni apposite.

`int()` — trasforma in numero intero

Ecco come `int()` converte valori di tipo diverso in un numero intero:

```
x = int(3.9)      # 3 - la parte decimale viene TAGLIATA (non arrotondata!)
y = int("42")     # 42 - una stringa numerica diventa un numero
z = int(True)     # 1
w = int(False)    # 0

print(x, y, z, w) # 3 42 1 0
```

:::caution `int(3.9)` restituisce `3`, non `4`. Il decimale viene tagliato, non arrotondato. Se vuoi arrotondare, usa `round(3.9)` che fa `4`. :::

`float()` — trasforma in numero decimale

Ecco come `float()` trasforma un valore in un numero con la virgola:

```
a = float(5)      # 5.0
b = float("3.14") # 3.14
d = float(True)   # 1.0

print(a, b, d)    # 5.0 3.14 1.0
```

`str()` — trasforma in testo

Questa è la conversione più usata, perché spesso devi mettere numeri dentro messaggi di testo:

```
eta = 16
# print("Ho " + eta + " anni")      # ERRORE - non puoi sommare str e int
print("Ho " + str(eta) + " anni")    # OK - "Ho 16 anni"

n = str(42)      # "42"
f = str(3.14)    # "3.14"
b = str(True)    # "True"
```

In alternativa alle conversioni manuali, puoi usare le **f-string** (stringhe formattate), che gestiscono tutto automaticamente:

```
eta = 16
print(f"Ho {eta} anni") # "Ho 16 anni" – più comodo!
```

`bool()` — trasforma in vero/falso

Ogni valore in Python può essere convertito in booleano. I valori "vuoti" o "zero" diventano `False`, tutto il resto diventa `True`:

```
print(bool(0))      # False – zero è falso
print(bool(""))     # False – testo vuoto è falso
print(bool([]))     # False – lista vuota è falsa
print(bool(None))   # False

print(bool(1))      # True
print(bool(-5))     # True – qualsiasi numero diverso da zero è vero
print(bool("ciao")) # True – qualsiasi testo non vuoto è vero
```

Cosa succede se la conversione non è possibile?

Se provi a convertire qualcosa che non ha senso, Python ti dà un `ValueError`:

```
int("ciao")      # ValueError – "ciao" non è un numero!
float("abc")     # ValueError
```

Per gestire questo caso senza far crashare il programma, puoi usare un blocco `try/except` (che vedremo in dettaglio nel capitolo sugli errori):

```
valore_inserito = "pippo"
try:
    numero = int(valore_inserito)
    print("Numero valido:", numero)
except ValueError:
    print("Quello che hai scritto non è un numero!")
```